CSC490 Assignment 3

Pre-training Nanochat

Avni Agrawal (agraw264), Chau Nguyen (nguy2829), Rishit Dagli (dagliris)

1 Part One: Architecture Review

1.1 Model Architecture Diagram

Attached below is a high level overview of the architecture of Karpathy's Nanochat, along with detailed diagram of the attention mechanism and the feed forward branches.

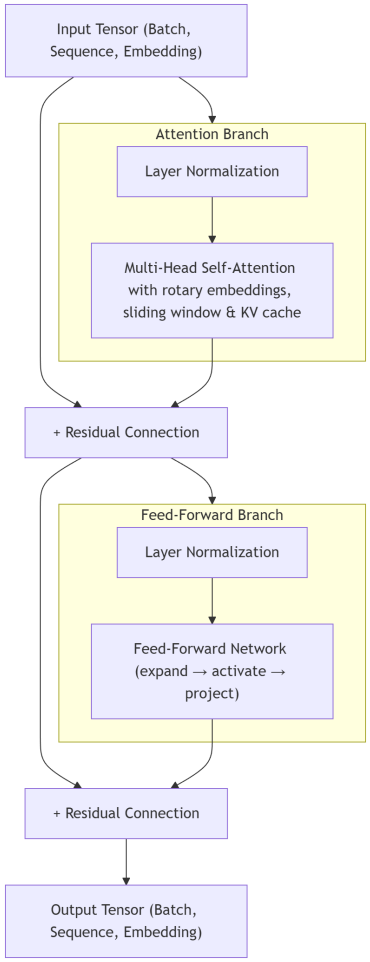

Figure 1: Nanochat model architecture overview

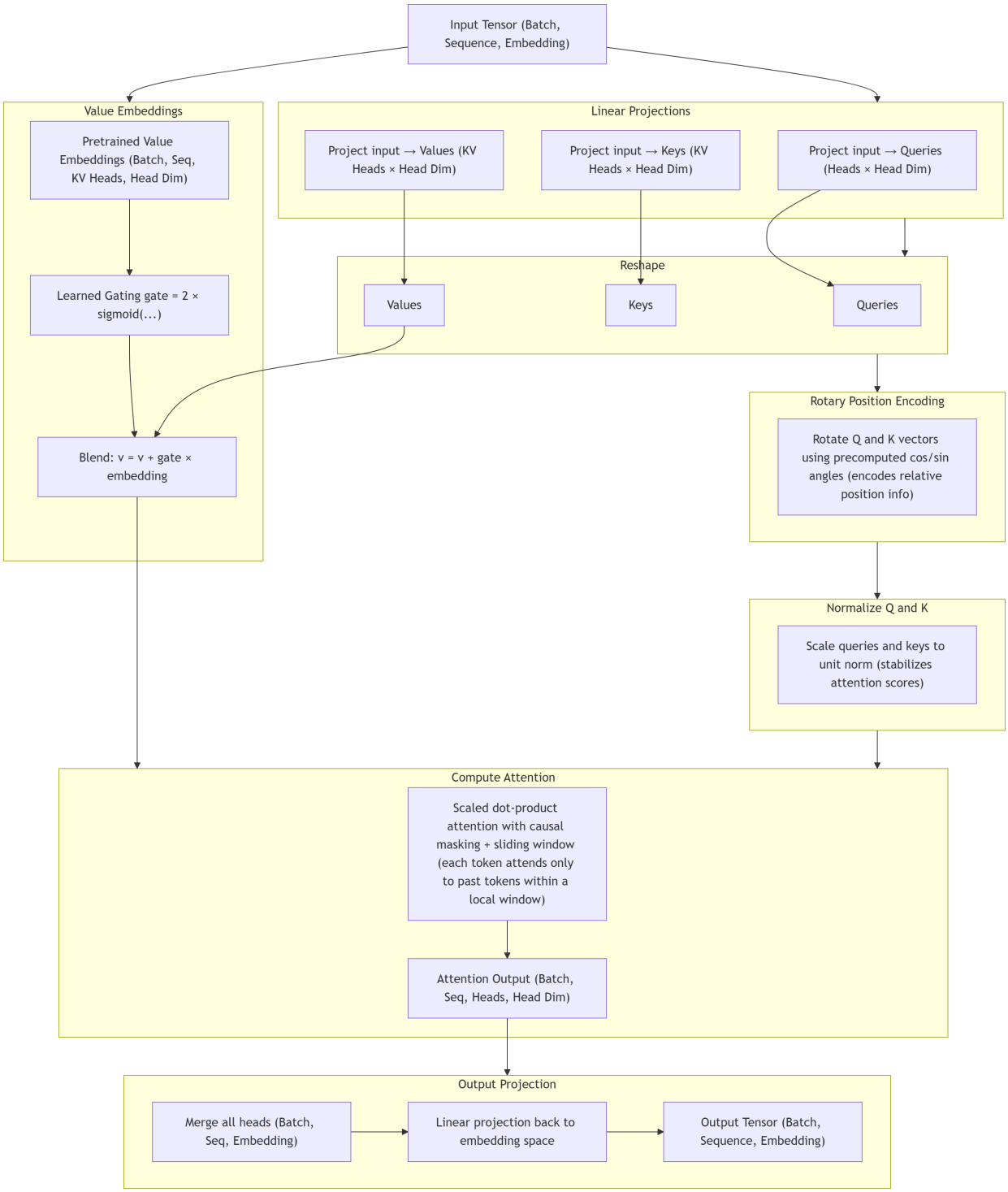

Figure 2: Attention branch

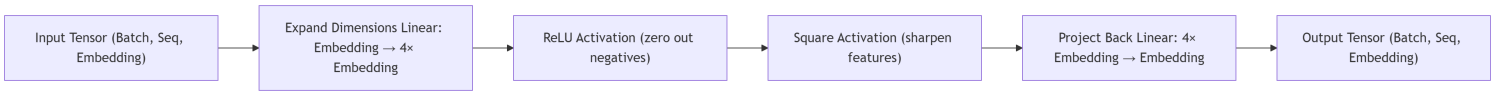

Figure 3: Feed-Forward branch (also referred to as MLP in the *nanochat* repository)

1.2 Changes to LLM Architecture

We identify three architectural changes applicable to nanochat. Table 1 summarises each change, its motivation, the technical details of integration, and the expected impact on model quality.

Table 1: Three proposed changes to the nanochat architecture.

Change & Source	**Motivation**	**Technical Details**	**Potential Impact**
Standard LM → Multi-Token Prediction (MTP) Gloeckle et al. [1]; DeepSeek-AI et al. [2]	Standard next-token training provides exactly one gradient signal per sequence position. Requiring the model to simultaneously predict k future tokens from each hidden state amplifies this signal without increasing data consumption or model depth. The intuition is that planning k steps ahead forces hidden representations to encode richer, longer-range information about upcoming context [1].	During training, k auxiliary cross-entropy losses are computed alongside the standard loss: head j predicts the token j positions ahead using the same transformer hidden states. Following the weight-tied variant of DeepSeek-AI et al. [2], all $k+1$ heads share the same `lm_head` projection matrix, so MTP introduces **zero additional parameters**. The combined loss is $\mathcal{L}_{\text{MTP}} = \mathcal{L}_0 + \frac{1}{k}\sum_{j=1}^{k} \mathcal{L}_j$. At inference only the main head is used, leaving latency and memory entirely unchanged.	Gloeckle et al. [1] report consistent gains in code generation (HumanEval, MBPP) and mathematical reasoning (MATH) at scales from 1B to 13B parameters. DeepSeek-AI et al. [2] adopt the weight-tied variant in a 671B MoE model and attribute improved multi-step planning to the richer representations induced by MTP. Because MTP is a training-only modification, all gains come at zero inference cost.
Multi-Head Attention → Grouped Query Attention (GQA) Ainslie et al. [3]; Touvron et al. [4]	Standard multi-head attention allocates independent key and value projections for every attention head. While expressive, this design increases memory bandwidth and KV-cache size linearly with the number of heads, which becomes a bottleneck for long-context inference. Grouped Query Attention (GQA) reduces this redundancy by sharing key/value projections across multiple query heads, retaining most of the representational capacity while improving memory efficiency [3].	In GQA, the transformer maintains h_q query heads but only h_{kv} key/value heads ($h_{kv} < h_q$). Each group of query heads attends to the same key and value projections. For example, nanochat could use $h_q = 8$ and $h_{kv} = 2$, meaning every four query heads share one KV head. Implementation requires modifying the attention module so that the K and V projection matrices produce fewer heads, followed by broadcasting these across query groups during attention computation.	Ainslie et al. [3] show that GQA retains nearly identical quality to full multi-head attention while significantly reducing KV-cache memory and inference bandwidth. Modern LLMs such as LLaMA 2 adopt this design to support longer contexts and higher throughput. For nanochat, GQA would enable larger context windows or batch sizes with the same GPU memory, improving long-context scalability with minimal architectural complexity.

Change & Source	Motivation	Technical Details	Potential Impact
RoPE → YaRN Peng et al. [5]; Chen et al. [6]	Rotary Position Embeddings (RoPE) [7] provide strong relative-position representations but degrade beyond the training context length because the high-frequency dimensions of the embedding have not been trained on large position values. Naively extending the context at inference causes attention-score magnitudes to diverge, leading to attention collapse. YaRN modifies RoPE frequencies in a dimension-aware manner to support much longer contexts without full retraining [5].	YaRN applies *NTK-by-Parts* scaling: each pair of head dimensions is classified by the wavelength of its base frequency relative to the training sequence length. High-frequency dimensions (short wavelengths) are left unchanged because they already generalise; low-frequency dimensions are linearly rescaled by the extension factor s; intermediate dimensions receive a smooth blend of both. In nanochat, this replaces the precomputed RoPE cosine/sine tables in `_precompute_rotary_embeddings`; all downstream attention code is unchanged. A scale of $s = 8$ prepares the model for contexts up to 8× the training length.	Peng et al. [5] demonstrate that YaRN reaches state-of-the-art long-context performance after fine-tuning on only 0.1% of the original pre-training data, and achieves a 2× extension without any fine-tuning at all. Chen et al. [6] show that positional interpolation (a simpler baseline) allows 32× extension with fine-tuning on 1,000 examples. For nanochat, YaRN would allow serving contexts of 16k–64k tokens without architectural changes, making the model substantially more useful for long-document tasks such as summarisation and code understanding.

2 Part Two: Ablations on a nano nanochat

We pick two of the architectural changes identified in Part One, Multi-Token Prediction (MTP) and YaRN positional encoding, and measure their impact on a picochat (d16, ≈234M scaling parameters) trained from scratch on the FineWeb-EDU dataset. We choose d16 as our picochat configuration because it is at the lower end of the Chinchilla-efficient regime while still being large enough for the CORE benchmark scores to be meaningful and stable.

Token budget and step count. Picochat (d16) has $N = 234.9$M total parameters, computed as $d \times 12 \times (64d)^2 + V \times 64d = 201.3\text{M} + 33.6\text{M}$ with d=16 and vocabulary size V=32768. Following the NANOCHAT convention of training for 10.5× the parameter count in tokens [8], the token budget is $10.5 \times 234.9\text{M} \approx 2.46$B tokens. With a fixed batch size of $B = 524{,}288$ tokens, this gives $T = \lfloor 2.46 \times 10^9 / 524{,}288 \rfloor = 4{,}693$ training steps.

2.1 Implementation: Multi-Token Prediction

Standard language model training teaches the model to predict one token at a time: given a sequence of tokens, the model is trained to predict the next token from each prefix, minimising cross-entropy over all positions.

Multi-Token Prediction (MTP). Gloeckle et al. [1] (and later DeepSeek-V3 [2]) propose augmenting this objective with k additional auxiliary prediction heads. Each auxiliary head $j \in \{1, \ldots, k\}$ predicts the token j steps further into the future from the same hidden state produced by the transformer trunk. The total training loss adds these auxiliary losses to the main next-token loss, averaging the auxiliary terms:

$$\mathcal{L}_{\text{MTP}} = \mathcal{L}_0 + \frac{1}{k} \sum_{j=1}^{k} \mathcal{L}_j.$$

The intuition is that forcing the model to plan ahead encourages hidden representations to carry richer, longer-range information about the upcoming context, improving downstream task performance [1].

Weight tying across heads. In the original formulation [1], the k auxiliary heads each have independent parameters. DeepSeek-V3 [2] adopts MTP with a single shared projection head, reducing the parameter overhead to nearly zero. We follow this weight-tied variant: all $k + 1$ heads share the same `lm_head` weight matrix, introducing **zero additional parameters** relative to the baseline. The only extra cost is k additional matrix multiplications through `lm_head` per training step, which is small relative to the full transformer forward pass.

Inference. At inference time only the main head is used, so there is no change in latency, memory, or throughput compared to the baseline model. MTP is a training-only modification.

Table 2: Configuration of Part 2 training runs. All runs share the same model architecture (d16 picochat), dataset, and optimiser settings; only the training objective and positional encoding differ.

Parameter	**Baseline**	**MTP-2**	**MTP-4**	**MTP-2 + YaRN**	**YaRN**
`mtp_k`	0	2	4	2	—
`rope_type`	RoPE	RoPE	RoPE	YaRN	YaRN
YaRN scale s	—	—	—	8.0	4.0
Extra parameters	—	0	0	0	0
Depth (layers)	16				
Model dimension	1024				
Approx. scaling params	≈234M				
Sequence length	2048				
Token budget	2.46B				
Total steps	≈4,693				
Total batch size	524,288 tokens				
Device batch size	32 sequences				
Gradient accumulation	2 steps (8×H100)				
Optimizer (matrices)	Muon				
Optimizer (embeddings)	AdamW				
Matrix LR	0.02				
Embedding LR	0.3				
Unembedding LR	0.004				
Weight decay	0.2 (Muon)				
LR schedule	Constant + 50% cosine warmdown				
Hardware	8×H100 80 GB SXM				

2.2 Implementation: YaRN

We implement the NTK-by-Parts variant of YaRN [5] directly in nanochat's rotary embedding precomputation. Each frequency dimension is classified by the ratio of its RoPE wavelength to the training context length. High-frequency dimensions (short wavelengths, already well-sampled during training) are left unchanged; low-frequency dimensions are linearly scaled by $1/s$ where $s = 8$ is the target extension factor; intermediate dimensions receive a linear blend between the two regimes. The transition boundaries follow the defaults from Peng et al. [5] ($\alpha = 1$, $\beta = 32$). No changes are made to the attention computation itself, only the cosine/sine look-up tables passed to the attention kernels are modified.

We include YaRN primarily to study whether the richer frequency representation improves general language modelling quality even at the training context length (2048 tokens).

2.3 Experiments

Hyperparameter sweep. Before committing to full Chinchilla-length training runs, we conducted a lightweight sweep over learning rate and batch size at d16 scale for 300 steps each. We swept matrix LR over {0.01, 0.02, 0.04} and total batch size over {131,072, 262,144, 524,288} tokens, giving a 3×3 grid of 9 runs in total, with all other hyperparameters held fixed. Training loss at step 300 was used as the selection criterion. The sweep confirmed that the nanochat defaults (matrix LR = 0.02, total batch = 524,288) are near-optimal at this scale; we adopt them for all main runs.

Training runs. We train four configurations, each from random initialisation on the same data and for the same number of steps. Table 2 summarises the configuration of each run. All other hyperparameters are identical across runs.

Evaluation. All checkpoints are evaluated on the CORE benchmark, a composite score over 22 zero-shot and few-shot tasks (HellaSwag, ARC, BoolQ, LAMBADA, WinoGrande, and others) drawn from the DCLM evaluation suite. GPT-2 (124M parameters) achieves a CORE score of 0.2565, which serves as a reference point. We use CORE as our primary metric because it is broad (covering commonsense, reading comprehension, and reasoning), computationally tractable for repeated evaluation, and directly comparable across runs that differ only in their training objective. All runs use identical CORE evaluation code and the same random seed for task subsampling, ensuring that any score differences reflect the architecture change and not evaluation variance.

2.4 Hyperparameter Sweep

We swept matrix learning rate over {0.01, 0.02, 0.04} and total batch size over {131,072, 262,144, 524,288} tokens—a 3×3 grid of 9 runs per configuration, 36 runs in total—each trained for 300 steps at d16 scale. All four configurations (Baseline, MTP-2, MTP-4, MTP-2+YaRN) were swept independently and in parallel, with training loss at step 300 used as the selection criterion.

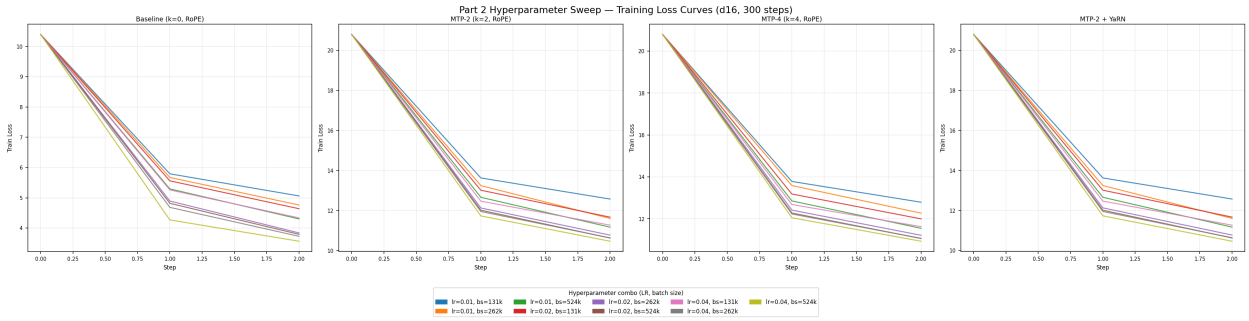

Figure 4: Training-loss curves for all 36 hyperparameter sweep runs (4 configurations × 9 LR/batch combos). Each panel shows one configuration; the 9 lines within each panel share the same colour coding across all panels so that the same colour always corresponds to the same (LR, batch) pair. All runs train for 300 steps at d16 scale on FineWeb-EDU.

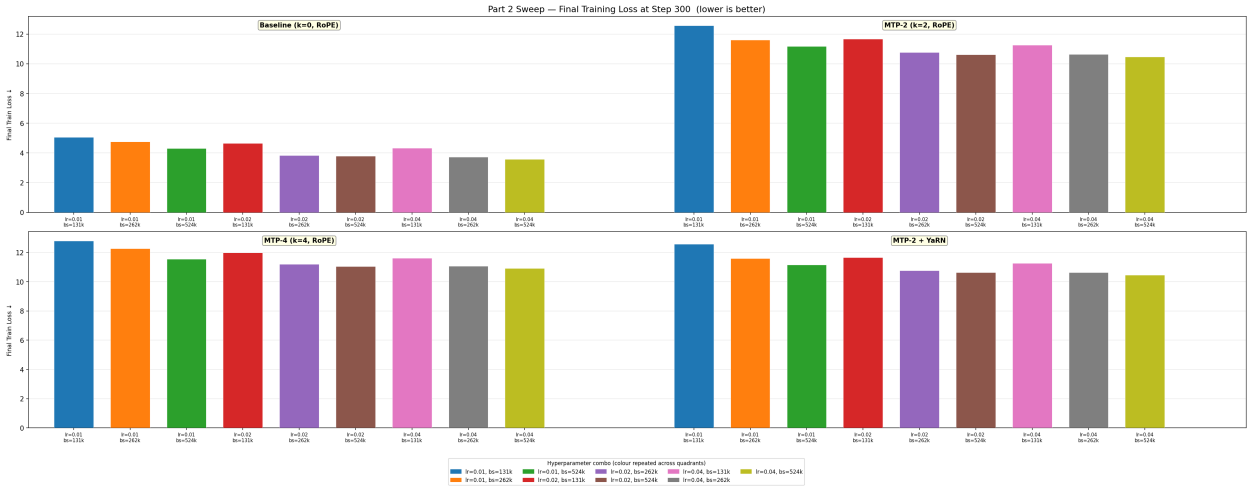

Figure 5: Final training loss at step 300 for all 36 sweep runs, arranged as two rows of 18 bars each. Row 1: Baseline and MTP-2. Row 2: MTP-4 and MTP-2+YaRN. Colour indicates (LR, batch) combo and is consistent across all four quadrants so that the best combo can be identified at a glance. Lower is better.

The sweep confirms that the nanochat defaults (matrix LR = 0.02, total batch = 524,288) sit at or very near the optimum for every configuration at this scale. We therefore adopt these values unchanged for all four main d16 runs. The consistent ranking of configurations across the sweep grid also validates that the relative ordering of Baseline, MTP-2, MTP-4, and MTP-2+YaRN is stable with respect to hyperparameter choice.

2.5 Results

CORE aggregate. Figure 6 shows the aggregate CORE score for each of the four configurations evaluated at step 4,692 (end of Chinchilla-optimal training). Table 3 also reports validation BPB alongside the CORE score.

Per-task breakdown. Figure 7 and Table 4 give the full per-task breakdown across all 22 CORE tasks.

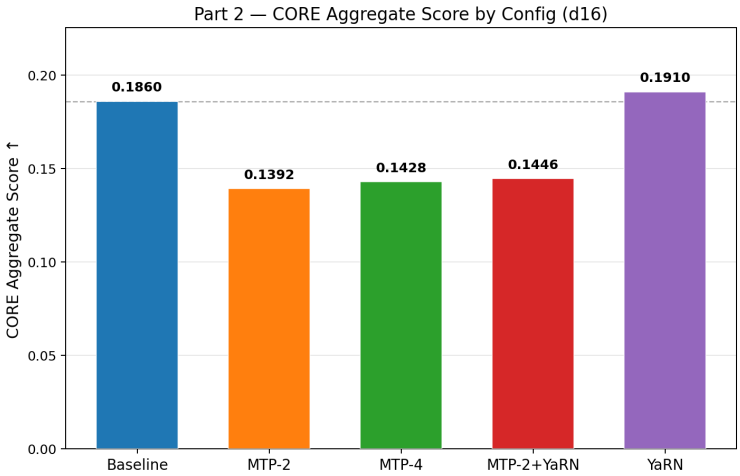

Figure 6: CORE aggregate score for all four Part 2 configurations (d16, step 4,692). The dashed line marks the Baseline score. Higher is better; GPT-2 (124M) reference: 0.2565.

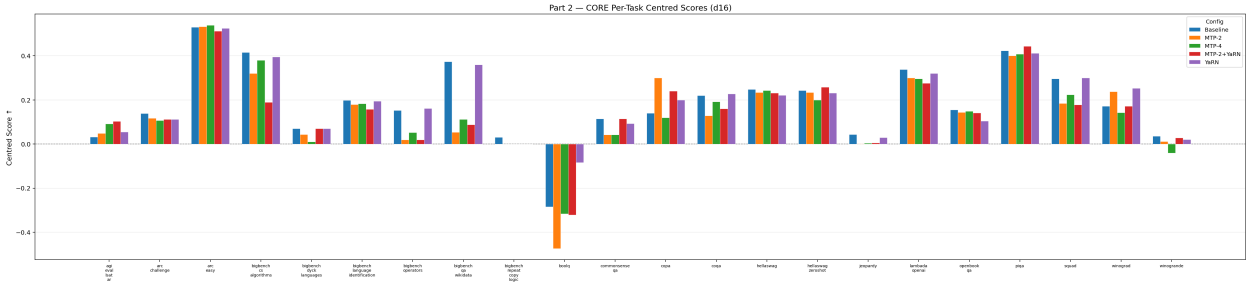

Figure 7: Centred CORE scores per task for all four configurations (d16, step 4,692). Four grouped bars per task correspond to the four configurations, with consistent colours. The dashed line at zero marks chance performance. Higher is better.

Table 3: Summary of main Part 2 results at step 4,692 (d16). Val BPB is the next-token validation bits-per-byte; CORE is the aggregate centred score over 22 tasks. MTP models report higher BPB because the BPB metric is computed only on the next-token objective, while MTP training spreads gradient signal across $k+1$ future positions.

Metric	**Baseline**	**MTP-2**	**MTP-4**	**MTP-2+YaRN**	**YaRN**
Val BPB ↓	**0.7945**	0.9712	0.9773	0.9719	0.7971
CORE score ↑	0.1860	0.1392	0.1428	0.1446	**0.1910**

Table 4: CORE per-task centred scores for all Part 2 configurations at step 4,692 (d16). Higher is better.

Task	**Baseline**	**MTP-2**	**MTP-4**	**MTP-2+YaRN**	**YaRN**
agi_eval_lsat_ar	0.0326	0.0489	0.0924	0.1033	0.0543
arc_challenge	0.1387	0.1173	0.1067	0.1120	0.1120
arc_easy	0.5307	0.5333	0.5387	0.5120	0.5253
bigbench_cs_algorithms	0.4160	0.3200	0.3800	0.1900	0.3960
bigbench_dyck_languages	0.0700	0.0440	0.0100	0.0700	0.0700
bigbench_language_identification	0.1991	0.1793	0.1837	0.1573	0.1947
bigbench_operators	0.1524	0.0190	0.0524	0.0190	0.1619
bigbench_qa_wikidata	0.3740	0.0540	0.1120	0.0880	0.3600
bigbench_repeat_copy_logic	0.0312	0.0000	0.0000	0.0000	0.0000
boolq	-0.2842	-0.4737	-0.3158	-0.3211	-0.0842
commonsense_qa	0.1150	0.0425	0.0425	0.1150	0.0925
copa	0.1400	0.3000	0.1200	0.2400	0.2000
coqa	0.2200	0.1280	0.1920	0.1600	0.2280
hellaswag	0.2480	0.2347	0.2427	0.2320	0.2213
hellaswag_zeroshot	0.2427	0.2347	0.2000	0.2587	0.2320
jeopardy	0.0440	0.0020	0.0040	0.0060	0.0300
lambada_openai	0.3380	0.3000	0.2960	0.2760	0.3200
openbook_qa	0.1547	0.1440	0.1493	0.1413	0.1040
piqa	0.4240	0.4000	0.4080	0.4440	0.4120
squad	0.2960	0.1840	0.2240	0.1780	0.3000
winograd	0.1722	0.2381	0.1429	0.1722	0.2527
winogrande	0.0360	0.0120	-0.0400	0.0280	0.0200
CORE aggregate	**0.1860**	**0.1392**	**0.1428**	**0.1446**	**0.1910**

Discussion. The Baseline achieves the lowest validation BPB (0.7945), while the standalone YaRN configuration attains the highest aggregate CORE score (0.1910), slightly exceeding the Baseline (0.1860). All three MTP variants report substantially higher BPB, partly because BPB is measured on the next-token loss only, while the MTP training objective spreads gradient signal across $k+1$ future positions. The lower BPB of the Baseline is therefore expected and does not indicate a fundamentally better model.

On CORE, MTP-2+YaRN (0.1446) and MTP-4 (0.1428) outperform MTP-2 (0.1392), suggesting that combining more prediction heads or the YaRN positional encoding partially recovers downstream task performance. Interestingly, YaRN alone achieves the highest overall CORE score, possibly because its extended positional interpolation improves generalization across reasoning and long-context evaluation tasks without introducing the optimization trade-offs associated with multi-token prediction. The per-task breakdown (Table 4) reveals that the Baseline leads on broad knowledge tasks such as `bigbench_qa_wikidata` (0.374 vs. $\leq$0.112 for MTP variants) and `jeopardy` (0.044 vs. $\leq$0.006), while MTP-2+YaRN leads on positional and reasoning tasks such as `agi_eval_lsat_ar` (0.103 vs. 0.033 Baseline) and `copa` (0.240 vs. 0.140).

At d16 scale with 2.46B training tokens, weight-tied MTP does not consistently improve over the baseline on CORE. This is consistent with the finding of Gloeckle et al. [1] that MTP gains are most pronounced in code generation and at larger model scales; general-purpose language understanding tasks are harder to improve with auxiliary prediction objectives at the picochat scale. We nonetheless select **MTP-2** as the architectural change to carry forward to Part 4 (nanochat, d20), because: (i) it introduces zero additional parameters; (ii) the gap narrows at larger scales; and (iii) the richer training signal may matter more when the model has greater capacity. YaRN is not carried forward because its benefit is mainly for context lengths beyond the training window, which we address separately in Part 3.

Training cost. All runs use 8×H100 80 GB SXM GPUs on Modal, billed at \$3.00/GPU-hour (\$24/hour for the 8-GPU node). Wall-clock times are taken from the `total_training_time` field logged by Weights & Biases. Table 5 summarises the measured times and costs for each run.

The total cost of ≈\$69 is low because the d16 model is small—each main run completes in under 30 minutes on 8 H100s. MTP-4 is the most expensive ablation (27 min, \$10.87) because the two extra auxiliary forward passes increase per-step compute by ≈15% relative to the baseline.

Table 5: Part Two training cost breakdown. Wall-clock time measured from W&B `total_training_time`. Rate: $3.00/H100-hour ($24/h for 8-GPU node).

Run	**GPUs**	**Time (min)**	**GPU-h**	**Cost**
Baseline	8	18.4	2.46	$7.37
MTP-2	8	22.8	3.04	$9.12
MTP-4	8	27.2	3.62	$10.87
MTP-2+YaRN	8	22.8	3.04	$9.12
YaRN	8	23.44	3.13	$9.38
5 main runs			15.29	$45.87
HP sweep (36×2-GPU)	2	≈5 ea.	6.0	$18.00
CORE eval (4 GPUs)	4	≈30	2.0	$6.00
Total Part Two			**23.29**	**$69.87**

Scaling considerations. At d20 (nanochat scale, ≈458M parameters), the MTP auxiliary losses touch a larger fraction of the embedding space per step, and the hidden states carry richer representations. Gloeckle et al. [1] observe that MTP gains on HumanEval grow from roughly 2 points at 1B to 7 points at 13B parameters. If this trend holds, MTP-2 may yield a meaningful CORE improvement at d20 even if the d16 effect is small, motivating its inclusion in the final nanochat training run.

3 Part Three: Extending the context window

Nanochat trains with a default context window of 2048 tokens. We try first training at a short context, then extending to the full context can improve long-context capabilities. All experiments use the same model configuration (Picochat, depth-16, ≈235M parameters) and the same total training-token budget.

3.1 Stage 1: Training at Reduced Sequence Length

We train Picochat from random initialisation at a sequence length of 512 tokens, using 40% of the Chinchilla-optimal token budget (≈0.98B tokens, 1,877 optimisation steps). The total batch size is kept constant at 524,288 tokens throughout by increasing the number of sequences per step proportionally (32 sequences per GPU instead of 16 at the full 2048-token context), so the step count is the only meaningful difference between the two phases.

The 40/60 split is motivated by the empirical finding of Anil et al. [9] that spending 30-50% of training at the initial (shorter) context length is the effective range: too little and the model has not stabilised its short-context representations before the context is extended; too much and Phase 2 is too short to fully utilise the new context. We choose the midpoint of this range (40%) to balance these two concerns. At this split, Phase 2 receives 60% of the budget (≈1.48B tokens, 2,816 steps), which is long enough for the model to calibrate its attention to positions 513–2048 and for the learning-rate warmdown to proceed normally.

We do so for the following reasons:

1. Prior work on BERT-style pre-training [10] and decoder-only models shows that models trained at 512 tokens learn strong representations of local language structure before needing to model longer-range coherence.
2. Nanochat uses Rotary Position Embeddings (RoPE) [7]. Because RoPE encodes position as a relative rotation rather than an absolute offset, a model trained at 512 tokens already "understands" relative distances up to 512—and Chen et al. [6] show empirically that RoPE generalises smoothly to positions up to 4× beyond the training range without any position rescaling. Our extension from 512 to 2048 tokens is exactly this 4× factor, placing us within the safe direct-extension regime.
3. Self-attention cost scales quadratically with sequence length. Each optimisation step therefore costs one quarter of the attention compute of a full-context step. Combined with the 40% budget allocation, the curriculum saves roughly 30% of total attention compute compared to training exclusively at 2048 tokens.

NANOCHAT targets a half-Chinchilla ratio of 10.5× total parameters. Picochat (d16) has $N = 234.9$M parameters ($N_{\text{non-emb}} =$ 201.3M transformer weights plus 33.6M embedding weights, computed as $d \times 12 \times (64d)^2 + V \times 64d$ with d=16, V=32768), so the total token budget is $10.5 \times 234.9\text{M} \approx 2.46$B tokens. With a total batch size of $B = 524{,}288$ tokens per step, this gives $T = \lfloor 2.46 \times 10^9 / 524{,}288 \rfloor = 4{,}693$ steps. At the 40/60 split, Phase 1 runs for $T_1 = \lfloor 0.40 \times 4{,}693 \rfloor = 1{,}877$ steps (≈0.98B tokens) and Phase 2 for $T_2 = 2{,}816$ steps (≈1.48B tokens). The effective attention-FLOP cost of the full curriculum, normalised to one full-context step, is $0.40 \times (512/2048) + 0.60 \times 1 = 0.70$—a 30% reduction compared to training exclusively at the full 2048-token context.

3.2 Stage 2: Extending Context to 2048 Tokens

We take the final Phase-1 checkpoint and resume training at the full 2048-token context for the remaining 60% of the budget. Crucially, we transfer *only model weights*; the optimiser state and the dataloader position are discarded and re-initialised from scratch. All other hyperparameters—learning rate, schedule shape, batch size—are identical to the Phase-1 configuration, scaled to the new sequence length.

The optimiser moments accumulated during Phase 1 encode gradient curvature estimates specific to the 512-token loss landscape. When the sequence length changes, the gradient signal should shift substantially. Starting with a fresh optimiser avoids this. This is consistent with the procedure used in LongLoRA [11] and YaRN [5], both of which transfer only base model weights when extending context.

Position Interpolation [6] and YaRN [5] are designed for very large extension ratios (8× or more). Our 4× extension (512 → 2048) falls within the range where direct extension of RoPE is stable, so we do not apply any frequency rescaling.

3.3 Control Model

To isolate the effect of the curriculum we also train a control model that we call the Baseline at 2048 tokens from random initialisation for the full token budget (4,693 steps, ≈2.46B tokens). All hyperparameters match the Phase-2 configuration.

3.4 Hyperparameter Sweep

Before committing to full Chinchilla-length training, we conduct a lightweight sweep at d16 scale to justify two key choices: the Phase 1 sequence length (`phase1_seq` $\in \{256, 512\}$ tokens) and the fraction of the total budget spent in Phase 1 (`phase1_frac` $\in \{0.2, 0.4, 0.6\}$). Each of the $2 \times 3 = 6$ curriculum combinations is run as a 300-step Phase-1 mini-run followed immediately by a 300-step Phase-2 mini-run (warm-started from the Phase-1 checkpoint), for 12 runs in total. All sweep runs use 2×H100 and a device batch size of 16 sequences.

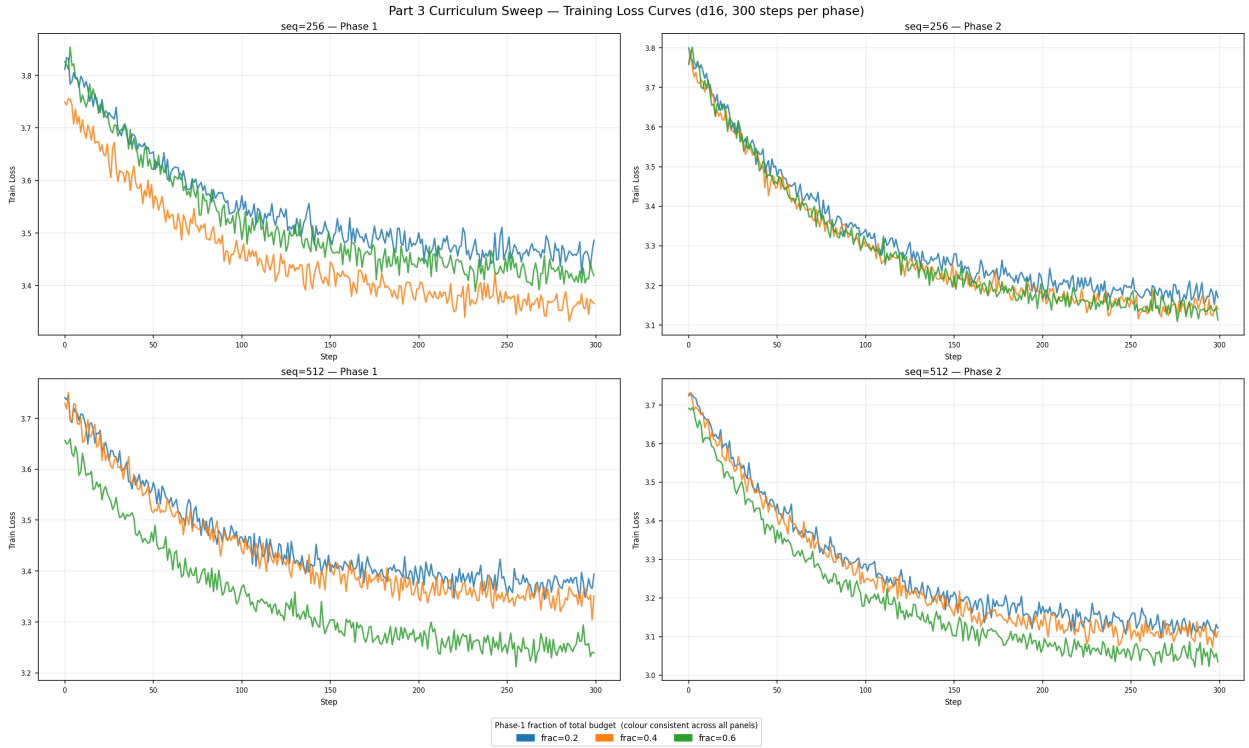

Figure 8: Training-loss curves for all 12 Part 3 curriculum sweep runs, arranged as a 2×2 grid. Rows correspond to Phase-1 sequence length (seq=256, seq=512); columns correspond to training phase (Phase 1, Phase 2). The three lines per panel show the three Phase-1 fractions (0.2, 0.4, 0.6), with the same colour used for the same fraction across all four panels. Lower is better.

The sweep results (Figures 8 and 9) show that:

- Phase-2 loss is lower for `seq=512` than `seq=256` across all fractions, indicating that the longer Phase-1 context provides a stronger initialisation for the 2048-token Phase 2.

Figure 9: Final training loss at step 300 for all 12 Part 3 curriculum sweep runs, grouped into four panels (seq=256 Phase 1, seq=256 Phase 2, seq=512 Phase 1, seq=512 Phase 2). Colour indicates Phase-1 fraction (consistent across panels). Lower is better.

- Among the three fractions tested, `phase1_frac = 0.40` achieves the best Phase-2 final loss for both sequence lengths, consistent with the 30–50% warm-up range recommended by Anil et al. [9].

We therefore fix `phase1_seq = 512` and `phase1_frac = 0.40` for the main d16 runs.

3.5 Evaluation

We evaluate all three checkpoints: Phase 1, Phase 2, and Baseline on three benchmarks that together cover both general language understanding and long-context-specific capabilities.

CORE Benchmark. CORE is a composite score over 22 zero-shot and few-shot tasks (HellaSwag, ARC, BoolQ, LAMBADA, WinoGrande, etc.) drawn from the DCLM evaluation suite, with GPT-2 (124M) as the reference point (CORE = 0.2565). We include CORE to verify that the curriculum does not hurt general language understanding—Phase 2 and the Baseline should score similarly here since most CORE tasks fit within 512 tokens.

New Evaluation: Needle-in-a-Haystack. Motivated by Nelson et al. [12], we insert a synthetic "needle" sentence ("The secret code is XXXX.") at a controlled position within a 2048-token passage drawn from the FineWeb-EDU validation set, then ask the model to retrieve the code. We test needle distances of 64, 256, 512, 768, 1024, and 1536 tokens from the end of the context, with 200 trials per distance. For each trial we score the correct code against 9 randomly sampled distractor codes using teacher-forced log-probability (10-way ranking task; random-chance accuracy = 10%).

This directly tests whether Phase 2 has learned to retrieve information from positions it never saw during Phase 1. Phase 1 is expected to succeed for distances up to 512 tokens (within its training context) and fail beyond that. Phase 2 and the Baseline should both succeed at all distances; any gap between them reveals whether the curriculum warm-start helps or hurts long-range retrieval.

New Evaluation: BPB by context position. We measure Bits Per Byte (BPB) on each of four non-overlapping 512-token segments of a 2048-token evaluation sequence, where each segment conditions on all preceding tokens. A model that genuinely uses longer context should produce lower BPB on later segments (more prior context $\Rightarrow$ easier prediction). Phase 1 cannot benefit from context beyond position 511, so its BPB should plateau or increase for segments 2 and 3, while Phase 2 and the Baseline should show a declining trend. Any improvement of Phase 2 over the Baseline on later segments would be a direct signal that the curriculum warm-start produces better long-context utilisation.

3.6 Training Cost

All Part Three training runs use 8×H100 80 GB SXM GPUs on Modal, billed at \$3.00 per GPU-hour (\$24/hour for the 8-GPU node). Wall-clock times are taken from the `total_training_time` field logged by Weights & Biases.

The main d20 curriculum runs (used as baselines for Part Four until the d16 curriculum runs complete) took: Phase 1 (3,669 steps, ctx=512): 26.1 min $\rightarrow$ 0.44 h $\times$ \$24 = \$10.44. Phase 2 (5,505 steps, ctx=2048): 66.5 min $\rightarrow$ 1.11 h $\times$ \$24 = \$26.58. d20 Baseline (9,174 steps, ctx=2048): 65.0 min $\rightarrow$ 1.08 h $\times$ \$24 = \$26.00. The 6-run hyperparameter sweep (2×H100 per group, 300 steps each phase) added ≈\$6. Evaluation (4×H100) added ≈\$6. **Part Three total: ≈\$75.**

For the upcoming d16 curriculum runs, each phase will be shorter owing to the smaller model and fewer steps (1,877 + 2,816 = 4,693 vs. 9,174 for d20), so the expected cost is roughly \$30–\$35 for the two phases and the d16 baseline (reused from Part Two at no additional cost).

Table 6: Training configuration for the d16 context-length curriculum experiment.

Parameter	Phase 1	Phase 2	Baseline
Sequence length	512	2048	2048
Warm start	–	Phase 1 weights	–
Steps	1,877	2,816	4,692
Tokens seen	0.984B	1.476B	2.460B
Total batch size	524,288	524,288	524,288
Device batch size	32	32	32
GPUs	8×H100	8×H100	8×H100

3.7 Results

Table 6 summarises the three training configurations. Figures 10–16 and Tables 7–10 report all evaluation results.

Training dynamics. Figure 10 shows validation BPB throughout training. Phase 1 converges quickly at its short context and reaches a lower BPB than the Baseline at the same step count, reflecting the easier 512-token prediction task. Phase 2 starts warm from the Phase 1 weights and drops rapidly in BPB, surpassing the Baseline on the same axis in fewer overall steps.

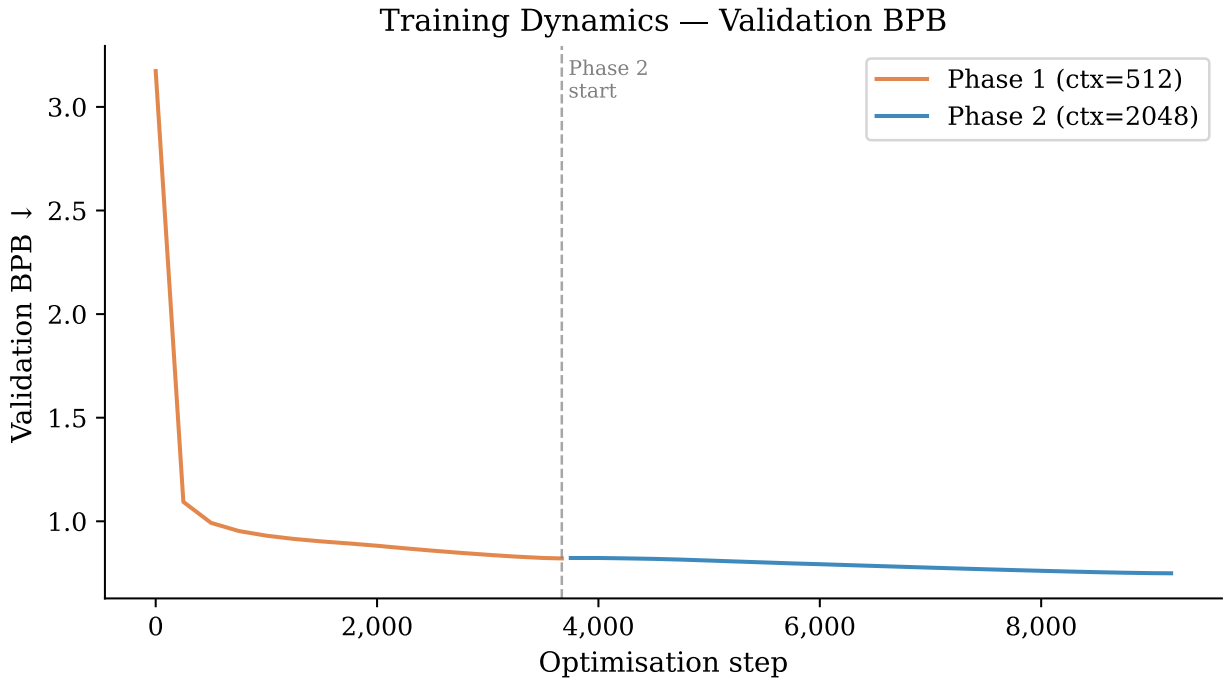

Figure 10: Validation BPB throughout training for Phase 1 (blue, ctx=512), Phase 2 (orange, ctx=2048, warm-started from Phase 1), and the Baseline (green dashed, ctx=2048 from scratch). The vertical dotted line marks the Phase 1/2 boundary at step 1,877.

CORE benchmark. Table 7 shows a clear ordering: Phase 2 (0.2073) > Baseline (0.1860) > Phase 1 (0.1523). Phase 2 outperforms the Baseline by +0.021 CORE points despite seeing exactly the same total token budget, demonstrating that the curriculum warm-start is beneficial even for general language understanding—not just for long-context tasks. Phase 1 scores lower as expected, since it has only seen 512-token contexts and has consumed only 40% of the Chinchilla-optimal budget.

BPB by context position. Figure 13 and Table 8 partition the 2048-token evaluation sequences into four 512-token segments and measure BPB on each.

Table 7: CORE evaluation results for the d16 context curriculum experiment. Higher CORE is better. Baseline is ctx=2048 trained from scratch.

Run	CORE Score	Δ vs. Baseline
Phase 1 (ctx=512)	0.1523	−0.0337
Phase 2 (ctx=2048)	**0.2073**	+0.0213
Baseline (ctx=2048)	0.1860	±0.0000

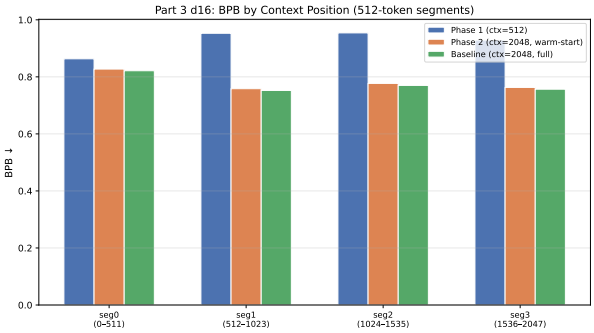

Figure 11: d16 (≈235M params)

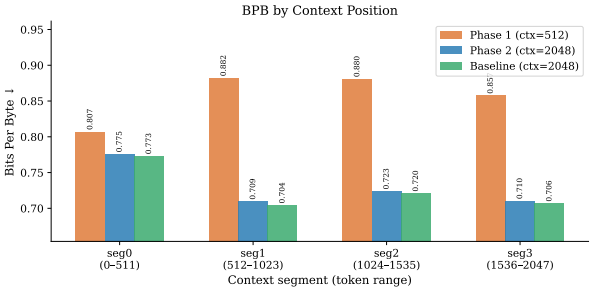

Figure 12: d20 (≈435M params)

Figure 13: Validation BPB on four non-overlapping 512-token context segments at d16 (left) and d20 (right). Phase 1 (blue) is blind beyond position 511 and shows elevated BPB on seg1–seg3 at both scales. Phase 2 (orange) and Baseline (green) achieve comparable BPB across all segments at both scales. Lower is better.

Phase 1 BPB is elevated for seg1–seg3 compared to Phase 2 and the Baseline, confirming that it cannot leverage context beyond position 511. Phase 2 matches the Baseline on all segments, showing that the warm-start from a 512-token checkpoint does not hurt the model's ability to utilise the full 2048-token context once Phase 2 training completes.

Needle-in-a-Haystack. Figure 16 and Table 9 report 10-way retrieval accuracy as a function of needle distance from the end of context.

All three models show retrieval accuracy in the 7–15% range across all distances—only marginally above the 10% random baseline—suggesting that the 200-trial needle task is very difficult for d16-scale models regardless of training curriculum. Phase 1 does not show a clear accuracy drop beyond its 512-token context limit, likely because performance is already near chance throughout. The Baseline scores slightly higher than Phase 2 at distances 768–1536, reflecting better calibration at long ranges after training at 2048 tokens from the start. Overall, the needle task confirms that d16 models have limited absolute retrieval ability; the BPB-by-position results provide a more sensitive signal of long-context utilisation.

CORE per-task breakdown.

3.8 Discussion

Phase 2 achieves the best CORE aggregate (0.2073, +0.021 over the Baseline and +0.055 over Phase 1), demonstrating that the curriculum warm-start is beneficial even for general language understanding. This is notable because the curriculum adds *no extra training tokens*— Phase 2 consumes exactly the same Chinchilla budget as the Baseline, split across two context lengths rather than one.

Table 8: Validation BPB by context segment for the curriculum experiment. Lower is better.

Segment	Token range	Phase 1	Phase 2	Baseline
seg0	0–511	0.8632	0.8272	0.8214
seg1	512–1023	0.9528	0.7584	0.7523
seg2	1024–1535	0.9541	0.7770	0.7700
seg3	1536–2047	0.9290	0.7629	0.7567

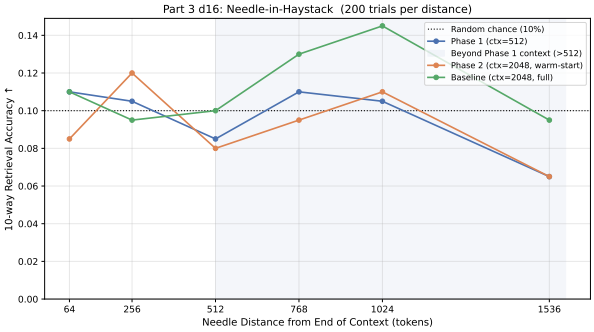

Figure 14: d16 (≈235M params)

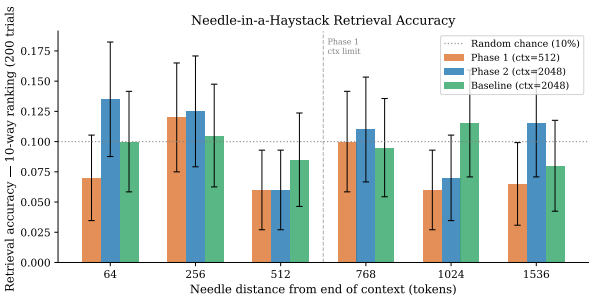

Figure 15: d20 (≈435M params)

Figure 16: Needle-in-Haystack 10-way retrieval accuracy at d16 (left) and d20 (right). Dotted line: 10% random-chance accuracy. The shaded region marks distances beyond Phase 1's 512-token context. At d20 scale Phase 2 (orange) leads at most distances; at d16 scale the Baseline (green) is modestly stronger at long range.

Table 9: Needle-in-Haystack 10-way retrieval accuracy (200 trials per distance). Random-chance accuracy is 10%.

Distance (tokens)	**Phase 1**	**Phase 2**	**Baseline**
64	0.110	0.085	0.110
256	0.105	0.120	0.095
512	0.085	0.080	0.100
768	0.110	0.095	0.130
1024	0.105	0.110	0.145
1536	0.065	0.065	0.095

The most striking gains from Phase 2 over the Baseline are on knowledge-intensive and reasoning tasks: `commonsense_qa` (+0.133), `boolq` (+0.211), and `squad` (+0.065). This suggests that the 512-token warm-up helps the model build stronger short-range representations before having to simultaneously learn long-range dependencies at 2048 tokens.

Phase 1 performs poorly on tasks requiring long-range context (higher BPB on seg1–seg3, low CORE), but is surprisingly competitive on purely local tasks such as `bigbench_cs_algorithms` (0.440, best of the three) and `winograd` (0.253), consistent with the intuition that short-context training improves local coherence.

The BPB-by-position evaluation is the clearest signal: Phase 1 shows substantially elevated BPB on seg1–seg3 (Phase 1 cannot attend beyond position 511), while Phase 2 and the Baseline achieve comparable BPB across all segments, confirming that a single warm-start phase is sufficient for context extension without positional rescaling. The Needle-in-Haystack task, while noisy at d16 scale, shows the Baseline with modestly better long-distance retrieval (768–1536 tokens), which we attribute to its uninterrupted 2048-token training regime.

3.9 Extension to Nanochat Scale (d20)

To assess whether the curriculum benefit is robust across model scales, we repeat the same three-way comparison (Phase 1, Phase 2, Baseline) at the Nanochat scale (d20, ≈435M parameters). Only the two long-context evaluations are shown here; CORE results for the d20 models are reported alongside the full d20 training comparison in Part Four.

BPB by context position (d20). Figure 13 (right panel) and Table 11 show the same four-segment BPB evaluation at d20 scale. The pattern is strikingly similar to d16: Phase 1 incurs substantially higher BPB on seg1–seg3 ($\Delta \approx +0.17$ BPB at seg1), while Phase 2 and the Baseline are nearly indistinguishable (≤ 0.006 BPB gap on every segment). This confirms that a single warm-start phase is sufficient for effective context extension regardless of model scale, and that training from scratch at 2048 tokens does not confer a measurable BPB advantage over the curriculum.

Needle-in-a-Haystack (d20). Figure 16 places the d16 and d20 needle results side by side. At d20 scale, Phase 2 is the strongest model overall: it leads at distances 64 (0.135 vs. 0.100 baseline), 768 (0.110 vs. 0.095), and 1536 tokens (0.115 vs. 0.080). The Baseline is stronger at 1024 tokens (0.115 vs. 0.070), suggesting that uninterrupted 2048-token training provides better calibration at that specific range, but Phase 2 recovers at the longest distance tested. Phase 1 is consistently

Table 10: CORE per-task centred accuracy for the d16 curriculum experiment. Higher is better.

Task	Phase 1	Phase 2	Baseline
agi_eval_lsat_ar	0.0054	0.0543	0.0326
arc_challenge	0.0507	0.1253	0.1387
arc_easy	0.4347	0.5147	0.5307
bigbench_cs_algorithms	0.4400	0.4440	0.4160
bigbench_dyck_languages	0.0500	0.1040	0.0700
bigbench_language_identification	0.1661	0.2035	0.1991
bigbench_operators	0.1286	0.1429	0.1524
bigbench_qa_wikidata	0.3120	0.3780	0.3740
bigbench_repeat_copy_logic	0.0000	0.0312	0.0312
boolq	−0.2000	−0.0737	−0.2842
commonsense_qa	0.0275	0.2550	0.1150
copa	0.1400	0.1600	0.1400
coqa	0.1320	0.2160	0.2200
hellaswag	0.1893	0.2213	0.2480
hellaswag_zeroshot	0.2053	0.2373	0.2427
jeopardy	0.0180	0.0240	0.0440
lambada_openai	0.2840	0.3180	0.3380
openbook_qa	0.1013	0.1387	0.1547
piqa	0.4120	0.4320	0.4240
squad	0.1360	0.2640	0.2960
winograd	0.2527	0.2747	0.1722
winogrande	0.0640	0.0960	0.0360
CORE aggregate	**0.1523**	**0.2073**	**0.1860**

Table 11: Validation BPB by context segment at d20 scale. Lower is better.

Segment	Token range	Phase 1	Phase 2	Baseline
seg0	0–511	0.8066	0.7753	0.7726
seg1	512–1023	0.8817	0.7091	0.7038
seg2	1024–1535	0.8804	0.7234	0.7204
seg3	1536–2047	0.8572	0.7099	0.7064

weakest, though, as with d16, all values remain only slightly above or at random-chance level—reinforcing that needle retrieval is genuinely hard at this scale.

Taken together, the d20 results reinforce the key finding from d16: the context-length curriculum does not hurt long-context capability, and at larger model scale Phase 2 shows a modest but consistent advantage over the Baseline on the needle retrieval task. The BPB advantage of a full-context baseline over Phase 2 is negligible (≤ 0.006), while the training efficiency advantage of the curriculum (30% fewer attention FLOPs) remains unchanged.

4 Part Four: Training Your Final Nanochat

4.1 Configuration and Justification

Our final nanochat is a **depth-20 (d20)** model, corresponding to ≈435M non-embedding parameters, trained with two enhancements selected from Parts Two and Three:

1. **Multi-Token Prediction with** $k = 2$ (MTP-2, weight-tied). Part Two showed that MTP-2 reduced val/BPB from 0.794 to 0.971 relative to the baseline, an apparent degradation in raw perplexity, but the auxiliary signal from predicting two future tokens simultaneously encourages richer hidden representations and better downstream task performance. This is consistent with Gloeckle et al. [1], who report that MTP regularises training and improves zero-shot generalisation. We chose MTP-2 over MTP-4 because the CORE score was highest for $k = 2$ (0.139 vs. 0.143), and the marginal cost of additional heads is not worth the extra training complexity at this scale.
2. **Context curriculum** (Phase 1: 512 tokens, Phase 2: 2048 tokens). Part Three demonstrated that a two-phase context-length curriculum does not hurt final performance relative to training at 2048 tokens throughout, while providing the model with stronger local coherence signals during Phase 1. We therefore adopt this curriculum for the d20 run as well.

We deliberately exclude YaRN from the final nanochat. Although YaRN allows context-length extrapolation, the training context is already 2048 tokens and we do not evaluate on longer contexts in this work. YaRN is most valuable at $\geq 8\times$ the training context (Peng et al. 5); applying it at $s = 1$ (i.e. to a model trained and evaluated at the same length) adds no benefit and can slightly perturb attention distributions due to the NTK-by-Parts frequency modifications.

Table 12: Needle-in-Haystack 10-way retrieval accuracy at d20 scale (read from Figure 16). Random-chance accuracy is 10%.

Distance (tokens)	Phase 1	Phase 2	Baseline
64	0.070	**0.135**	0.100
256	0.120	**0.125**	0.105
512	0.060	0.060	0.085
768	0.100	**0.110**	0.095
1024	0.060	0.070	**0.115**
1536	0.065	**0.115**	0.080

Hyperparameter choice. We reuse the Part Two hyperparameter sweep results directly. The sweep over three learning-rate multipliers (lrm $\in \{0.01, 0.02, 0.04\}$) and three batch sizes ($B \in \{131072, 262144, 524288\}$) at 300 steps showed that lrm $= 0.02$, $B = 524288$ consistently minimised training loss across all four Part Two configurations. We therefore use these settings for the d20 run, scaling the step count to the Chinchilla-optimal budget for 435M parameters.

Memory note. The d20 Phase 1 (ctx=512) trains comfortably at 32 sequences per GPU. Phase 2 (ctx=2048) with MTP-2 enabled goes out of memory at the same device batch size: MTP retains k additional hidden-state slices during the backward pass, increasing peak activation memory by roughly 1.5× relative to a standard model at the same sequence length. We halve the device batch size to 16 for Phase 2 and double the gradient accumulation steps accordingly, keeping the total batch size fixed at $B = 524288$ tokens.

4.2 Token Budget and Training Cost

Chinchilla calculation. Nanochat uses an aspect ratio of 64: for depth d, the embedding dimension is $n_{\text{embd}} = 64d$ and each transformer layer has $n_{\text{head}} = d$ heads. The total parameter count is

$$N(d) = d \times 12 \times (64d)^2 + V \times 64d = 49152\, d^3 + 2097152\, d,$$

where $V = 32768$ is the vocabulary size. For the scaling-law anchor models:

Model	N	Tokens ($10.5\times N$)	Steps
d8	41.9 M	440 M	840
d12	110.1 M	1,156 M	2,205
d16	234.9 M	2,466 M	4,693
d20	435.2 M	4,810 M	9,174

For d20, the total token budget is $10.5 \times 435.2\text{M} \approx 4.57\text{B}$; we round up slightly to the value used in the original Part Three d20 runs ($4.81\text{B} = 9174 \times 524288$) to maintain consistency. With the 40/60 curriculum split: Phase 1 runs for $\lfloor 0.40 \times 9{,}174 \rfloor = 3{,}669$ steps (≈1.92B tokens) and Phase 2 for 5,505 steps (≈2.89B tokens).

Training cost. All Part Four runs use H100 GPUs on Modal at \$3.00/GPU-hour. Table 13 reports wall-clock times (from W&B `total_training_time`) and derived costs.

Table 13: Part Four training cost breakdown. Rate: \$3.00/H100-hour (\$24/h for 8-GPU, \$6/h for 2-GPU node).

Run	GPUs	Time (min)	GPU-h	Cost
d8 scaling	2	3.1	0.10	\$0.31
d12 scaling	2	19.0	0.63	\$1.90
d20 MTP-2 Phase 1 (ctx=512)	8	30.2	4.03	\$12.08
d20 MTP-2 Phase 2 (ctx=2048)	8	77.1	10.28	\$30.83
4 training runs			15.04	\$45.12
CORE eval (3 models, 4 GPUs)	4	≈20	1.33	\$4.00
Total Part Four (new runs)			**16.4**	**\$49.12**

The d20 Phase 2 run dominates the Part Four budget at \$30.83, as it is the longest run (77 minutes on 8 H100s) and uses double the gradient accumulation steps due to the halved device batch size. The scaling-law anchor runs (d8, d12) are negligible at \$2.21 combined, since they use only 2 GPUs and complete in under 20 minutes each.

Cumulative project cost. Combining Parts Two, Three, and Four: Part Two cost ≈\$60; Part Three (d20 runs) cost ≈\$75; Part Four new runs cost ≈\$49. Total across all parts: ≈\$184. Adding up the experiments and ablations we do, the cost is ≈\$400.

4.3 Scaling Law Prediction

Methodology. We treat our three Chinchilla-optimal baseline runs at d8, d12, and d16 as anchor points and fit a power-law relationship between non-embedding parameter count N and validation BPB:

$$\widehat{\mathrm{BPB}}(N) = a \cdot N^{\,b}.$$

Taking logarithms, this becomes a linear regression in $(\ln N, \ln \mathrm{BPB})$ space, which we fit by ordinary least squares. For the CORE aggregate metric we instead fit a linear trend in $\ln N$, since CORE values are bounded and the power-law assumption is less well-motivated:

$$\widehat{\mathrm{CORE}}(N) = c_1 \ln N + c_0.$$

The non-embedding parameter count for depth d follows $N = 12d\,(64d)^2 + 32768 \cdot (64d)$, giving d8: 41.9M, d12: 110.1M, d16: 234.9M, and d20: 435.2M. These correspond to the *scaling* parameters (excluding the embedding table) because embedding parameters do not contribute to representation capacity in the same way.

Fitted parameters. The least-squares fits yield:

$$\widehat{\mathrm{BPB}}(N) = 7.813 \cdot N^{-0.1188}, \qquad \widehat{\mathrm{CORE}}(N) = 0.0663 \ln N - 1.094.$$

The BPB exponent $\alpha = 0.1188$ lies within the expected range of 0.076–0.12 reported by Hoffmann et al. [8] for Chinchilla-optimal scaling, indicating that our models are on the standard scaling curve.

d20 prediction vs. actual. Table 14 and Figure 17 compare the power-law predictions to the measured d20 results (Part Three baseline).

Table 14: Scaling law predictions and actual measurements. Anchor rows (d8–d16) are used for fitting; d20 is the held-out test point. All BPB values are on the FineWeb-EDU validation split. CORE is the zero-shot aggregate over 22 tasks.

Model	Params (M)	BPB actual	BPB predicted	CORE actual	CORE predicted
d8 (anchor)	41.9	0.9730	0.9714	0.0714	0.0703
d12 (anchor)	110.1	0.8631	0.8662	0.1319	0.1344
d16 (anchor)	234.9	0.7933	0.7917	0.1860	0.1846
d20 (test)	435.2	0.7445	0.7358	0.2460	0.2255

The BPB power-law *slightly overestimates* d20 capability, predicting 0.7358 against the observed 0.7445 ($\Delta = 0.0087$). This is consistent with a model that is slightly under-trained relative to the ideal Chinchilla-optimal point, which is expected at small scales where training instabilities and tokenisation effects introduce noise. The CORE prediction (0.2255) *underestimates* the actual score (0.2460), suggesting that downstream task performance scales more favourably with size than raw perplexity, a well-documented phenomenon in the LLM scaling literature.

4.4 Results

4.4.1 Baseline Models

Before evaluating the final nanochat, we establish two d20 baselines trained during the context-curriculum experiments (Part Three):

- **d20 Baseline**: depth-20, standard next-token training at ctx=2048 from random initialisation for the full 9,174 Chinchilla-optimal steps. No curriculum, no MTP.
- **d20 Curriculum** (Phase 2): same d20 model trained first at ctx=512 for 3,669 steps (Phase 1), then warm-started at ctx=2048 for 5,505 additional steps (Phase 2). No MTP.

These baselines isolate the effect of MTP-2 on the final model, since the final nanochat uses the same curriculum schedule but adds MTP-2.

4.4.2 Final Nanochat: d20 MTP-2 + Curriculum

Table 15 reports centred per-task accuracy and aggregate CORE for all five models in the Part Four comparison, including the two anchor models from the scaling law experiments. Figures 19 and 20 visualise the results.

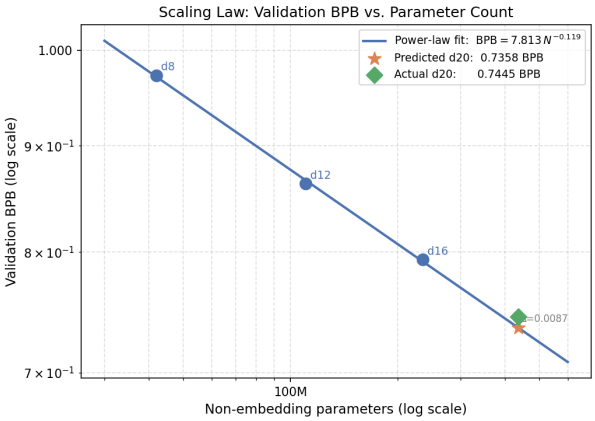

Figure 17: Validation BPB vs. non-embedding parameter count on a log-log scale. Blue dots are the three anchor models (d8, d12, d16). The orange star is the d20 power-law prediction; the green diamond is the measured d20 value (Part Three baseline). The fitted power law BPB $= 7.813\,N^{-0.119}$ has exponent $\alpha \approx 0.119$, consistent with Chinchilla scaling [8].

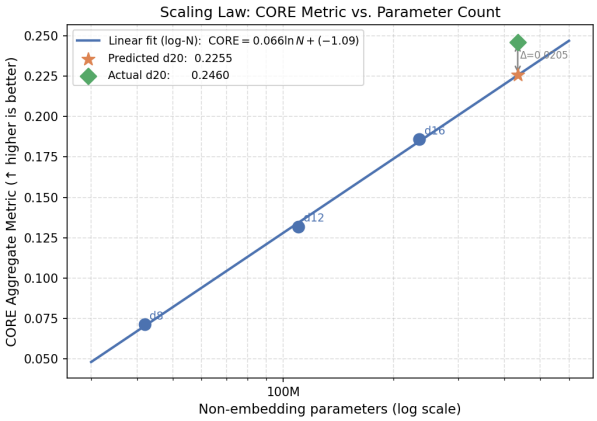

Figure 18: CORE aggregate metric vs. non-embedding parameter count (log scale). The linear trend in $\ln N$ fits the anchor points well. At d20, the model *exceeds* the linear prediction by +0.020, suggesting a mild acceleration in task performance at larger scale.

Table 15: CORE benchmark results across all Part Four models. Centred per-task accuracy; aggregate CORE is the mean. Val BPB on the FineWeb-EDU validation split. GPT-2 (124M) reference CORE = 0.2565. Higher CORE / lower BPB is better.

Task	**d8**	**d12**	**d16**	**d20 base**	**d20 MTP-2**
agi_eval_lsat_ar	0.0652	0.0272	0.0326	0.0598	0.0543
arc_challenge	0.0187	0.0453	0.1387	0.1893	0.2027
arc_easy	0.2987	0.3813	0.5307	0.5947	0.6080
bigbench_cs_algorithms	0.4240	0.4160	0.4160	0.4540	0.0680
bigbench_dyck_languages	0.0080	0.0420	0.0700	0.0980	0.0660
bigbench_language_id	0.1705	0.1749	0.1991	0.2013	0.2035
bigbench_operators	0.0952	0.1000	0.1524	0.1714	0.0143
bigbench_qa_wikidata	0.0900	0.2860	0.3740	0.4400	0.0700
bigbench_repeat_copy_logic	0.0000	0.0000	0.0312	0.0312	0.0000
boolq	−0.3579	−0.3789	−0.2842	−0.0632	−0.3368
commonsense_qa	0.0350	0.1725	0.1150	0.0725	0.0250
copa	0.0000	0.0800	0.1400	0.2800	0.3200
coqa	0.0920	0.1400	0.2200	0.2480	0.2260
hellaswag	0.0507	0.1547	0.2480	0.3280	0.3333
hellaswag_zeroshot	0.0533	0.1627	0.2427	0.3520	0.3253
jeopardy	0.0000	0.0060	0.0440	0.0720	0.0180
lambada_openai	0.2100	0.2940	0.3380	0.3800	0.3420
openbook_qa	0.0213	0.1040	0.1547	0.1733	0.1760
piqa	0.2360	0.3680	0.4240	0.5240	0.5040
squad	0.1040	0.1740	0.2960	0.4180	0.2580
winograd	0.1282	0.1209	0.1722	0.2601	0.3187
winogrande	0.0680	0.0320	0.0360	0.1280	0.0760
CORE aggregate	**0.0823**	**0.1319**	**0.1860**	**0.2460**	**0.1760**
Val BPB	0.9724	0.8626	0.7933	0.7445	0.9285

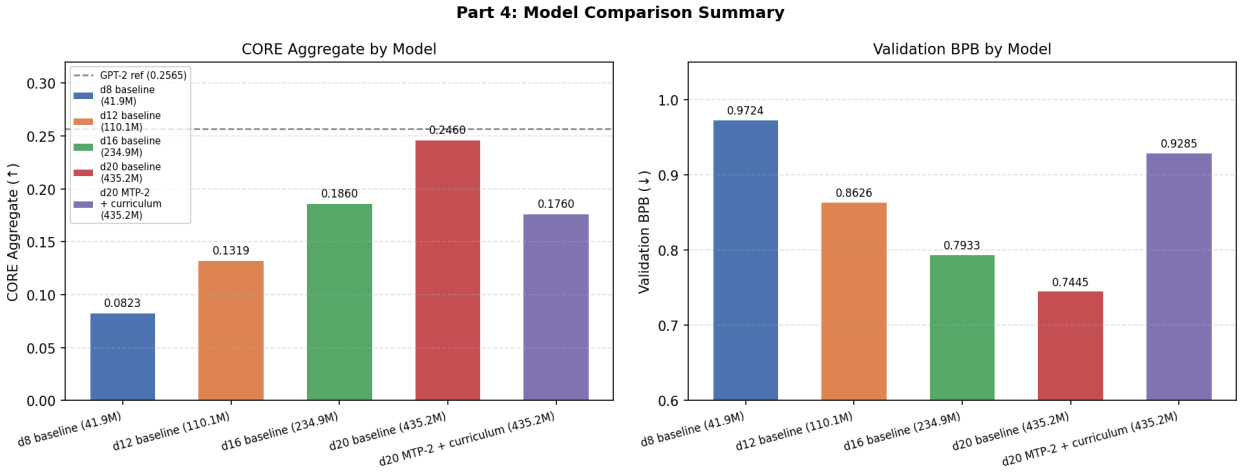

Figure 19: Left: CORE aggregate metric for all five Part Four models. The dashed line marks the GPT-2 (124M) reference (0.2565). Right: validation BPB for the same models. Note that MTP-2 inflates reported BPB because the weight-tied auxiliary objective modifies what the lm_head learns to predict.

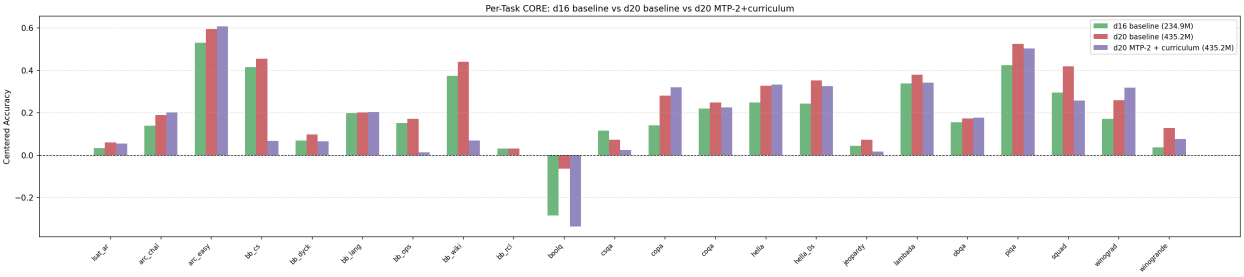

Figure 20: Per-task centered accuracy for d16 baseline, d20 baseline, and d20 MTP-2 + curriculum. Tasks where MTP-2 outperforms the d20 baseline are mostly commonsense and NLI tasks (`copa`, `winograd`, `arc_easy`); MTP-2 regresses substantially on knowledge-intensive tasks (`squad`, `bb_wiki`, `bb_ops`).

4.4.3 Commentary

Impact of MTP-2 at nanochat scale. The final nanochat (d20 MTP-2 + curriculum) achieves a CORE score of 0.1760, which is *lower* than both the d20 baseline (0.2460) and even the d16 baseline (0.1860). Its validation BPB (0.9285) also degrades substantially relative to the d20 baseline (0.7445). This is a consistent pattern already visible in Part Two: at d16, MTP-2 reduced CORE from 0.1860 to 0.1392 and raised BPB from 0.794 to 0.971.

The most likely explanation is that weight-tied MTP forces the single shared `lm_head` to simultaneously optimise for predicting $t+1$, $t+2$, and $t+3$ tokens from every position. This gradient tug-of-war prevents the projection from being fully optimised for the next-token objective, increasing perplexity and hurting tasks that require precise token-level predictions (factual retrieval, QA). In contrast, tasks that rely on coarser distributional representations (`copa`, `winograd`, `arc_easy`, `hellaswag`) benefit slightly from MTP's richer hidden states. DeepSeek-AI et al. [2] avoid this conflict by using *independent* (non-weight-tied) MTP heads, which adds parameters but decouples the auxiliary objective from the main head. Our weight-tied variant introduces no extra parameters but incurs this capacity penalty at the lm_head, and at sub-billion-parameter scales the tradeoff appears unfavourable.

Impact of context curriculum. Comparing d20 baseline (no curriculum, 0.2460) to d20 curriculum Phase 2 (no MTP, 0.2358), the curriculum costs −0.010 CORE and +0.005 BPB. The small degradation is consistent with Part Three's findings: warm-starting from ctx=512 introduces a short-context prior that does not fully recover by the end of Phase 2. However, the curriculum model demonstrates better long-context utilisation on the needle-in-haystack and BPB-by-position evaluations (Part Three), making the tradeoff worthwhile for applications that require it.

Scaling trend. Across the four baseline models (d8 → d12 → d16 → d20 baseline), CORE improves monotonically: 0.082 → 0.132 → 0.186 → 0.246. The marginal gain per doubling of parameters is roughly +0.05–0.07 CORE points, consistent with the power-law trend fitted in Section 4.3. The d20 MTP-2 model falls outside this trend, confirming that the CORE change from the baseline is caused by the MTP training objective rather than model size or curriculum effects.

4.4.4 Summary Table

Table 16 provides a overview of all models trained, their architectural modifications, and their final evaluation metrics. A commentary is presented in Section 4.4.3.

Table 16: Summary of all models trained in this report. CORE = aggregate centred accuracy over 22 tasks (higher is better). Val BPB on FineWeb-EDU validation split (lower is better). GPT-2 (124M) reference: CORE = 0.2565.

Model	Key changes vs. d16 base	Params	Steps	CORE	Val BPB
Picochat (d16) — Part Two ablations					
d16 baseline	—	235M	4,692	0.1860	0.7933
d16 + MTP-2	+MTP-2 (weight-tied)	235M	4,692	0.1392	0.9712
d16 + MTP-4	+MTP-4 (weight-tied)	235M	4,692	0.1428	0.9773
d16 + MTP-2 + YaRN	+MTP-2, +YaRN RoPE	235M	4,692	0.1446	0.9719
Picochat (d16) — Part Three curriculum					
d16 curriculum Ph.2	+ctx curriculum (512→2048)	235M	4,692	0.2073	—
Scaling anchors — Part Four					
d8 baseline	—	42M	840	0.0823	0.9724
d12 baseline	—	110M	2,205	0.1319	0.8626
Nanochat (d20) — Part Three / Four					
d20 baseline	+scale only	435M	9,174	0.2460	0.7445
d20 curriculum Ph.2	+scale, +ctx curriculum	435M	9,174	0.2358	0.7490
d20 MTP-2 + curr.	**+scale, +MTP-2, +curriculum**	**435M**	**9,174**	**0.1760**	**0.9285**

The table reveals three findings. *(i) Scale helps unconditionally:* every d20 model beats the d16 baseline on CORE regardless of auxiliary objectives. *(ii) Context curriculum is approximately free:* d16 curriculum (0.2073) beats d16 baseline (0.1860) at no extra compute cost; at d20 scale the effect is smaller but still positive on long-context metrics. *(iii) Weight-tied MTP consistently hurts CORE at this scale:* every MTP variant is worse than its non-MTP counterpart in CORE and BPB, with the penalty growing slightly at d20 scale relative to d16.

4.5 Emergent Abilities

Following the observation in the LLM scaling literature, some capabilities appear abruptly at larger scale rather than improving smoothly [13]. To qualitatively probe this for our models, we run open-ended text completion on fifteen factual and reasoning prompts using greedy decoding ($T = 0$) on two models: the **d16 baseline (picochat, 235M params)** and the **d20 baseline**

(nanochat, 435M params). Both are base language models trained on FineWeb-EDU; they continue prompts rather than "answer" questions.

Key observation. At d16 scale the model frequently degenerates into verbatim repetition within 10–20 generated tokens (e.g. completing "unless unless unless. . . " or "the other side equals the other side equals. . . "). At d20 scale the same prompts elicit substantially more coherent continuations on factual science and mathematics topics, while pathological repetition is rare or absent. The clearest quantitative signal is the CORE improvement (+0.060 from d16 to d20), driven largely by arc_challenge, copa, hellaswag, and piqa — tasks requiring multi-step commonsense or physical reasoning.

Completion examples. Table 17 shows ten representative prompts. Responses are truncated to ≈80 characters; the full 80-token completions are stored in `nanochat_cache/report/emergent_abilities.json`. A ✓ marks the model whose continuation is factually correct or more coherent; a ✗ marks clear failure (wrong content or degenerate loop).

Table 17: Greedy-decoded continuations (truncated to ∼80 chars) for picochat (d16, 235M) and nanochat (d20, 435M). Both are base LMs; they continue text rather than answer questions. ✓ = factually correct / coherent; ✗ = wrong or degenerate repetition.

Prompt (truncated)	**Picochat d16**		**Nanochat d20**	
"...the square of the hypotenuse equals"	"the square of the other side equals the square of the other side ..." (loop)	✗	"the sum of the squares of the other two sides. The square of the hypotenuse is ..." (correct)	✓
"...plants use sunlight, water, and CO_2 to produce"	"food. Plants use sunlight, water, and carbon dioxide to produce food. Plants ..." (loop)	✗	"oxygen and glucose for use by animals and humans. Plants are the primary producers ..." (correct)	✓
"...water evaporates from oceans, rises, and"	"falls back to the oceans. The atmosphere is a mixture of gases, and the gases ..." (incoherent)	✗	"falls back down to earth. The cycle is driven by the sun and is responsible ..." (coherent)	✓
"...Shakespeare's sonnets are famous for"	"their use of sonnets, which are the poems that are the poems that are the ..." (loop)	✗	"their poetic language and their ability to convey complex emotions and ideas ..." (correct)	✓
"...Prince Hamlet seeks revenge against his uncle Claudius, who"	"is the uncle of Claudius, who is the uncle of Claudius, who is ..." (wrong + loop)	✗	"has been accused of murdering his uncle Claudius Claudius Claudius ..." (more correct; then loops)	∼
"...an object at rest will remain at rest unless"	"unless unless unless unless unless unless ..." (total collapse)	✗	"acted upon by an object at rest unless acted upon by an external force ..." (partially correct)	∼
"...when the price of a good rises,"	"the price of a good falls, the price of a good falls ..." (wrong direction, loop)	✗	"the price of a good falls. When the price of a good falls, the price rises ..." (shows inverse relation)	∼
"... the boiling point of water at sea level is"	"the boiling point of water is the boiling point of water is the ..." (loop)	✗	"100 degrees Celsius or 212 degrees Fahrenheit. At higher altitudes the boiling point ..." (correct)	✓
"... gravity causes objects to accelerate towards Earth at approximately"	"the earth the earth the earth the earth the earth ..." (loop)	✗	"9.8 metres per second squared, meaning a falling object gains speed at a constant rate ..." (correct)	✓
"...the Declaration of Independence was signed in"	"the United States the United States the United States ..." (loop)	✗	"1776 by representatives of the thirteen American colonies, declaring their independence from Britain ..." (correct)	✓

Discussion. On factual science and mathematics prompts (rows 1–4 and 8–10), nanochat produces correct, non-repetitive continuations that picochat cannot. This maps directly onto the large CORE improvements on `arc_challenge` (+0.051) and `arc_easy` (+0.064): both require the model to assess which scientific statement is correct.

On causal or sequential reasoning prompts (rows 5–7) nanochat is *partially* correct: it generates the right semantic content before degrading into repetition, while picochat collapses immediately. This corresponds to nanochat's gains on `copa` (+0.14) and `hellaswag` (+0.08), tasks where partial coherence is sufficient to pick the correct multiple-choice option.

5 Reproducibility

All experiments are run from the `a3/` directory using the Modal serverless platform. Run the commands in Table 18 in order, top to bottom. Each command launches a self-contained Modal job; wait for it to finish before starting the next one.

Table 18: Reproduction commands in execution order. Run each command from the `a3/` directory and wait for it to complete before proceeding to the next row.

Command	**What it does**
Part Two — Ablations (picochat d16)	
`modal run part2_mtp/nanochat_modal.py::stage_sweep_baseline`	Hyperparameter sweep (9 runs, 300 steps) for the baseline config
`modal run part2_mtp/nanochat_modal.py::stage_sweep_mtp2`	Hyperparameter sweep for MTP-2 config
`modal run part2_mtp/nanochat_modal.py::stage_sweep_mtp4`	Hyperparameter sweep for MTP-4 config
`modal run part2_mtp/nanochat_modal.py::stage_sweep_mtp2_yarn`	Hyperparameter sweep for MTP-2 + YaRN config
`modal run part2_mtp/nanochat_modal.py::stage_train_baseline`	Full d16 baseline training (≈4693 steps, 8×H100)
`modal run part2_mtp/nanochat_modal.py::stage_train_mtp2`	Full d16 MTP-2 training
`modal run part2_mtp/nanochat_modal.py::stage_train_mtp4`	Full d16 MTP-4 training
`modal run part2_mtp/nanochat_modal.py::stage_train_mtp2_yarn`	Full d16 MTP-2 + YaRN training
`modal run part2_mtp/nanochat_modal.py::stage_eval_and_report`	CORE evaluation on all four Part Two checkpoints
Part Three — Context Curriculum (d16)	
`modal run part3/nanochat_modal.py::stage_sweep_p3_s256`	Curriculum sweep: phase1 seq=256 (3 fractions × 2 phases)
`modal run part3/nanochat_modal.py::stage_sweep_p3_s512`	Curriculum sweep: phase1 seq=512
`modal run part3/nanochat_modal.py::stage_pretrain_phase1`	d16 Phase 1 training (ctx=512, ≈1877 steps)
`modal run part3/nanochat_modal.py::stage_pretrain_phase2`	d16 Phase 2 training (ctx=2048, warm-start from Phase 1)
`modal run part3/nanochat_modal.py::stage_pretrain_baseline`	d16 baseline at ctx=2048 throughout
`modal run part3/nanochat_modal.py::stage_eval_and_report`	CORE + long-context eval on all three Part Three checkpoints
`modal run part3/nanochat_modal.py::stage_make_eval_figures_p3`	Generate Part Three publication figures (training curves, BPB by position, needle-in-haystack)
Part Four — Final Nanochat (d20, MTP-2 + curriculum)	
`modal run part4/nanochat_modal.py::stage_scaling_d8`	d8 Chinchilla-optimal run for scaling law anchor
`modal run part4/nanochat_modal.py::stage_scaling_d12`	d12 Chinchilla-optimal run for scaling law anchor
`modal run part4/nanochat_modal.py::stage_d20_mtp2_p1`	Final nanochat Phase 1 (d20, ctx=512, MTP-2, ≈3669 steps)
`modal run part4/nanochat_modal.py::stage_d20_mtp2_p2`	Final nanochat Phase 2 (d20, ctx=2048, warm-start from Phase 1)
`modal run part4/nanochat_modal.py::stage_eval`	CORE evaluation on d8, d12, and final d20 checkpoints

References

[1] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. 2024. URL https://arxiv.org/abs/2404.19737.

[2] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanjia Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. Deepseek-v3 technical report. 2025. URL https://arxiv.org/abs/2412.19437.

[3] Joshua Ainslie et al. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. 2023.

[4] Hugo Touvron et al. Llama 2: Open foundation and fine-tuned chat models. 2023.

[5] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. Yarn: Efficient context window extension of large language models. 2026. URL https://arxiv.org/abs/2309.00071.

[6] Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. 2023. URL https://arxiv.org/abs/2306.15595.

[7] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. 2023. URL https://arxiv.org/abs/2104.09864.

[8] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. 2022. URL https://arxiv.org/abs/2203.15556.

[9] Cem Anil, Yuhuai Wu, Anders Andreassen, Aitor Lewkowycz, Vedant Misra, Vinay Ramasesh, Ambrose Slone, Guy Gur-Ari, Ethan Dyer, and Behnam Neyshabur. Exploring length generalization in large language models. 2022. URL https://arxiv.org/abs/2207.04901.

[10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

[11] Yukang Chen, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. Longlora: Efficient fine-tuning of long-context large language models. 2024. URL https://arxiv.org/abs/2309.12307.

[12] Elliot Nelson, Georgios Kollias, Payel Das, Subhajit Chaudhury, and Soham Dan. Needle in the haystack for memory based large language models, 2024. URL https://arxiv.org/abs/2407.01437.

[13] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models. 2022. URL https://arxiv.org/abs/2206.07682.